Americal Sign Language (ASL) Alphabet Classification using Deep Learning A comparative study of ResNet-50 and MobileNetV2

Individual Report – Fyrooz Khan

An overview of the project and an outline of the shared work:
I completed the initial sections till methodology of the main report, preprocessed the data, developed the app and ran both the models. Then prepared a rough report on the mobilenetv2 model (pilot survey) to get a detailed idea and took report of the demo app performance results of both the models.

Introduction

American Sign Language (ASL) serves as a primary mode of communication for a significant portion of the Deaf and Hard-of-Hearing (DHH) population in the United States. While historical estimates frequently cited figures around 500,000 individuals, recent empirical research suggests that ASL usage is more widespread, with an estimated 1 million DHH adults using sign language (Rosen et al., 2022). Despite its prevalence, a significant communication barrier persists between signing and non-signing populations. Automated vision-based ASL recognition systems offer a promising technological bridge, enabling real-time interpretation through standard camera hardware without specialized sensors.

This project focuses on developing a deep learning system to classify static American Sign Language (ASL) alphabet images using the Kaggle ASL Alphabet dataset, which contains 26,000 labeled images across 26 classes, including A-Z (Alsharif et al., 2023; Lum et al., 2020). Because the dataset consists of still images, the problem is most appropriately framed as an image-classification task rather than a sequence-learning problem. This makes convolutional neural networks and transfer learning especially suitable for the project.

The project will compare two pretrained deep learning models: ResNet-50 and MobileNetV2. These models were selected because they represent two important goals of the project. ResNet-50 is included as a strong benchmark for classification accuracy, while MobileNetV2 is included as a lightweight and efficient model that is practical for deployment in a Streamlit application (Alsharif et al., 2023; Lum et al., 2020). The final demo will allow users to upload a still ASL hand image or capture one through a webcam and receive a predicted class label.

Background

Early sign language recognition systems relied heavily on sensor-based approaches, such as data gloves and motion capture devices, which impose significant hardware constraints and are impractical for everyday use. Vision-based approaches, leveraging standard cameras, have largely supplanted these methods.

Convolutional neural networks (CNNs) have become the dominant paradigm for image classification tasks. Transfer learning—where a model pre-trained on a large dataset (e.g., ImageNet) is adapted to a target task—has further accelerated progress by reducing the need for massive task-specific labeled datasets. MobileNetV2 (Sandler et al., 2018) represents a particularly attractive architecture for this setting: it achieves competitive accuracy at a fraction of the parameter count of larger models, making it suitable for deployment on edge devices and web applications.

Recent work by Zhang & Jiang (2024) provides a comprehensive review of deep learning for sign language recognition, highlighting that while dynamic gesture recognition (sequence modeling) remains an active frontier, static handshape classification—the focus of this work—has achieved near-perfect performance on clean, controlled datasets. The integration of hand landmark detection frameworks such as MediaPipe (Lugaresi et al., 2019) enables more robust real-world deployment by automating hand region localization.

Literature Review

Recent literature shows that deep learning has become one of the most effective approaches for ASL recognition, particularly for static alphabet classification tasks. CNN-based models are widely used because they learn visual features directly from image data and can distinguish subtle differences in hand shape more effectively than many traditional handcrafted-feature methods (Alsharif et al., 2023). This is especially important for ASL alphabet recognition, where several signs appear visually similar and require detailed feature extraction.

Among the available deep learning models, ResNet-50 and MobileNetV2 are especially relevant to this project. Alsharif et al. (2023) found that ResNet-50 achieved very high performance on a large ASL alphabet dataset, making it a strong benchmark model for accuracy. MobileNetV2 has also shown strong performance in ASL classification while using fewer parameters and lower computational cost, which makes it well suited for lightweight applications and practical deployment (Lum et al., 2020). Together, these studies support the use of ResNet-50 as a high-performance reference model and MobileNetV2 as an efficient deployment-oriented model.

Project Motivation

The motivation for this project is both practical and academic. From a practical standpoint, ASL recognition systems can contribute to accessibility by helping reduce communication barriers between Deaf or Hard of Hearing individuals and hearing non-signers (Alsharif et al., 2023). Building a model that can recognize static ASL alphabet images and demonstrate predictions through a Streamlit interface makes the project useful, understandable, and relevant to real-world assistive technology applications.

From a project design perspective, choosing only ResNet-50 and MobileNetV2 keeps the work focused and realistic for a short development timeline. ResNet-50 provides a literature-supported benchmark for strong predictive performance, while MobileNetV2 offers a more efficient architecture that is easier to deploy in an interactive demo. Using only these two models

creates a clear project story: compare a high-accuracy model with a lightweight practical model, then use the results to justify the final deployment choice.

2. Initial Dataset and Pilot Results

2.1 Source Datasets

Two publicly available ASL image datasets were used. The Kaggle ASL Alphabet dataset (Akash, 2018) contains 87,000 images across 29 classes (A–Z plus del, space, nothing) at 200×200 px (~3,000 images/class). The Mendeley ASL Dataset (Benabbas, 2022) contains 26,000 images across 26 classes (A–Z only) at 296×296 px, with tightly framed, consistently lit images (1,000 images/class). A pilot experiment training MobileNetV2 on the Kaggle dataset alone achieved high training accuracy but poor real-time inference. Root-cause analysis identified four systematic issues in the Kaggle data:

- **Wide-angle framing:** the hand occupies a small portion of each image, causing domain mismatch with tightly-cropped inference inputs.
- **Dark silhouettes / low contrast:** hand texture invisible to the network.
- **Label noise:** 'D' contains D, O, and F handshapes; 'X' shows an incorrect pose.
- **Hidden features:** M, N, and T show only the back of the hand, concealing the defining thumb placement.

All photos were compared with the standard ASL image available online.

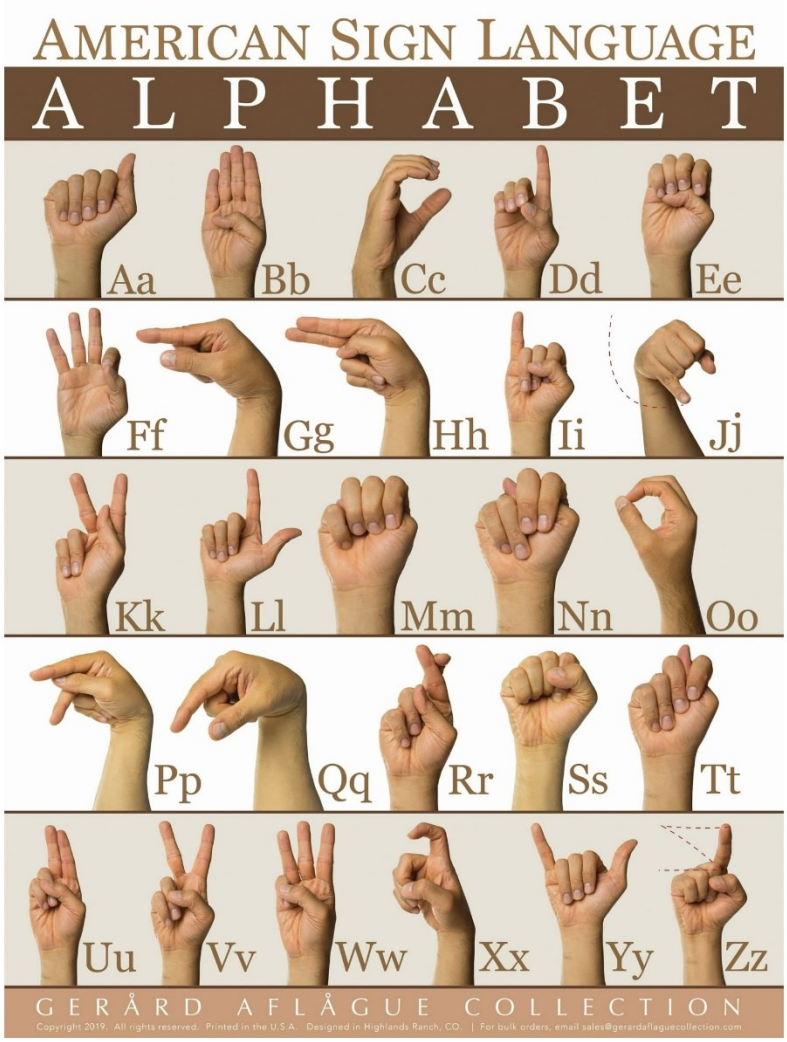


Figure: Reference Photos for Image Classification
Source: (Gerard Aflague Collection, n.d.)

3. Dataset Standardization Pipeline

3.1 Complementary Dataset — Mendeley ASL (26K)

The Mendeley ASL Dataset (Benabbas, 2022) was identified as a high-quality complement: 26,000 images, 26 letter classes (A–Z only), 296×296 px, tightly hand-framed with consistent centering and adequate lighting (1,000 images per class). The model was rerun on this dataset entirely and this resulted in high accuracy as well, but due to lack of varied images, the real-time results were not accurate.

So next, it was decided to merge the datasets from the two sources in the following way:

A three-phase pipeline was built to audit, clean, and merge both sources.

3.2 Pipeline Overview

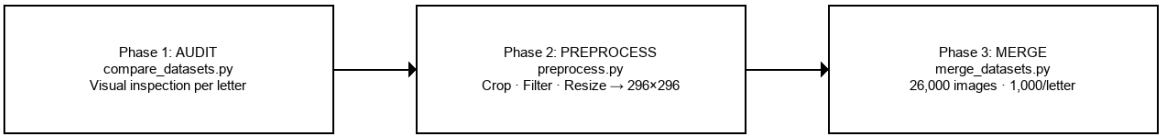
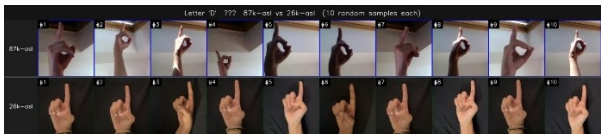


Figure 1. Three-phase dataset standardization pipeline.

3.3 Phase 1 — Visual Audit

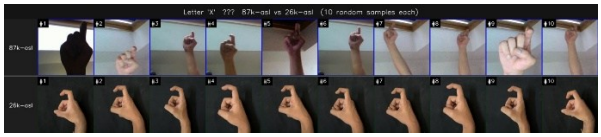
compare_datasets.py randomly sampled 10 images per letter from each dataset (seed=42) and generated side-by-side grids. Each letter was assigned a merge strategy:

Finding	Letters	Strategy
Same sign, compatible angle	A B C E F H I K L O Q R S U V W Y	Merge 500+500
87K hides defining thumb/finger feature	M N T	26K only
Label contamination / wrong pose	D X	26K only
Handedness or orientation mismatch	G P Z	26K only
26K uses left hand (ASL: right hand)	J	87K only

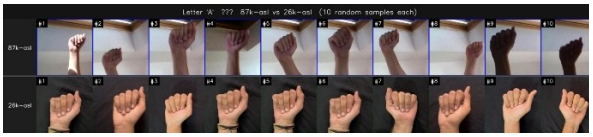


D — label contamination

M — hidden features



X — wrong pose



A — compatible

Figure 2. Audit grids (Row 1: 87K, Row 2: 26K). D — label contamination; M — hidden thumb feature; X — wrong pose; A — compatible (safe to merge).

3.4 Phase 2 — Preprocessing the 87K Dataset

preprocess.py applied a three-stage quality filter to every 87K image (8 parallel workers). Images failing any stage were discarded.

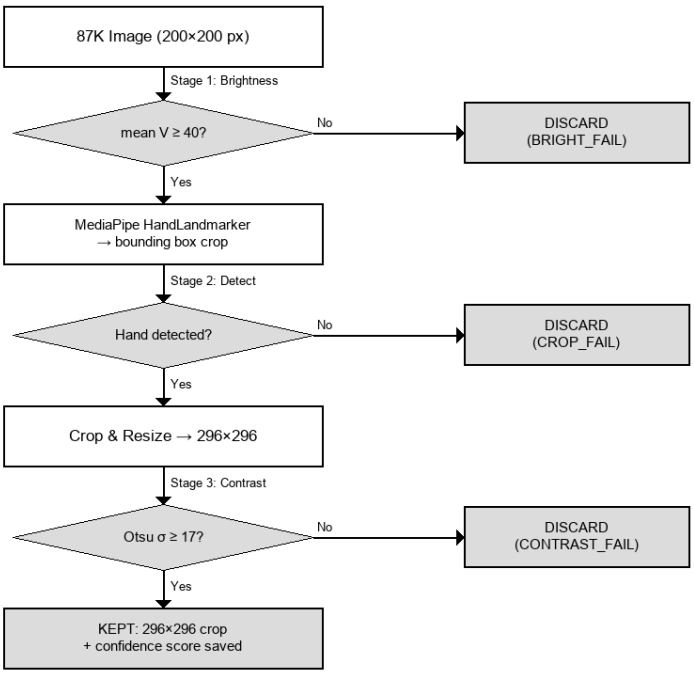


Figure 3. Three-stage preprocessing pipeline for 87K images.

Status	Count	%
Successfully kept	**49,614**	**57.0%**
No hand detected (CROP_FAIL)	23,409	26.9%
Low contrast (CONTRAST_FAIL)	13,977	16.1%
Brightness below threshold	0	0.0%
Total input	**87,000**	**100%**

3.5 Phase 3 — Merge

merge_datasets.py combined preprocessed 87K images (confidence ≥ 0.7) with 26K images using the per-letter strategies above. Uniform random sampling (seed=42) was used throughout to ensure reproducibility. Final dataset: 26,000 images — exactly 1,000 per letter (A–Z), perfectly balanced (17 letters merged · 8 from 26K only · 1 from 87K only).

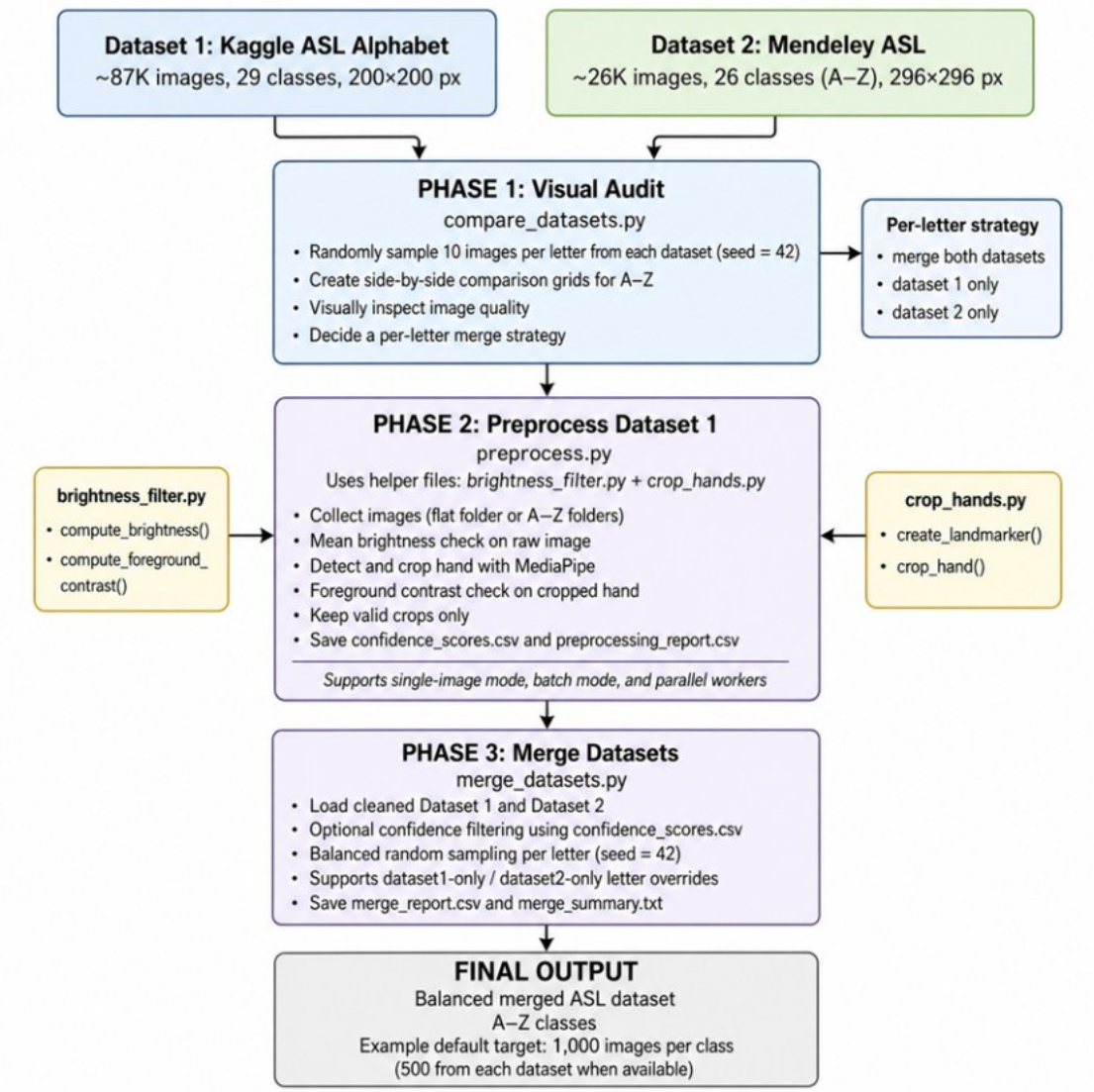


Figure 1. Three-phase dataset standardization pipeline.

Initial Pilot (MobilenetV2 model) on 26000 Dataset Setup and Results:

5. Experimental Design, Materials, and Methods

5.1 System Environment

All experiments were conducted using Python 3.10, TensorFlow 2.x / Keras, and scikit-learn.

5.2 Data Preprocessing and Augmentation

Images were preprocessed using the MobileNetV2-specific preprocessing function (mobilenet_v2.preprocess_input), which scales pixel values to the [−1, 1] range as required by the MobileNetV2 backbone. All images were resized to 224×224×3 (RGB).

To improve generalization and mitigate overfitting, the following augmentation operations were applied on-the-fly to training images using Keras's ImageDataGenerator:

Augmentation	Parameter

Rotation	±15°
Zoom	10%
Width shift	10%
Height shift	10%
Shear	10%
Brightness	[0.8, 1.2]
Horizontal flip	Disabled (signs are directional)

Table 2. Data augmentation parameters applied during training.

Horizontal flipping was deliberately disabled because ASL hand signs are inherently directional; mirrored images would represent a different sign or an invalid gesture. Validation and test images received only the preprocessing normalization, without any augmentation.

5.3 Model Architecture

The classification model is built on the MobileNetV2 backbone (pre-trained on ImageNet, weights='imagenet', include_top=False), with a custom classification head appended:

- Global Average Pooling 2D — reduces spatial feature maps to a 1D vector.
- Dropout (rate=0.3) — regularization to prevent overfitting.
- Dense (128 units, ReLU activation) — intermediate feature extraction.
- Dropout (rate=0.2) — additional regularization.
- Dense (26 units, Softmax activation) — final class probability output.

The total model has 2,425,306 trainable parameters after fine-tuning. The lightweight MobileNetV2 backbone (originally ~3.4M params) was selected for its balance of accuracy and computational efficiency, enabling deployment in resource-constrained inference environments.

5.4 Training Strategy

Training was executed in two phases to progressively adapt the pre-trained backbone to the ASL domain:

Phase 1 — Classifier Head Training: The MobileNetV2 base was frozen (base_model.trainable = False). Only the custom head was trained for up to 10 epochs using Adam optimizer at a learning rate of 1e-3. This phase rapidly fitted the new classification layers to the ASL feature space.

Phase 2 — Fine-Tuning: The base model was partially unfrozen (layers 100 onwards were set trainable). Training continued for up to 10 additional epochs at a reduced learning rate of 1e-5, allowing the top layers of the backbone to adapt to ASL-specific visual patterns.

Across both phases, the following callbacks were used:

- EarlyStopping (monitor=val_loss, patience=5, restore_best_weights=True) — prevents overfitting.
- ReduceLROnPlateau (monitor=val_loss, factor=0.5, patience=2) — adaptive learning rate decay.
- ModelCheckpoint (monitor=val_accuracy, save_best_only=True) — saves the best model to best_asl_model.keras.

The loss function was categorical cross-entropy and the batch size was 32 throughout training.

5.5 Training Hyperparameters Summary

Hyperparameter	Phase 1	Phase 2
Optimizer	Adam	Adam
Learning rate	1e-3	1e-5
Batch size	32	32
Max epochs	10	10
Base model layers frozen	All (153)	First 100
Loss function	Categorical crossentropy	Categorical crossentropy
Early stopping patience	5	5
LR reduction factor	0.5 (patience 2)	0.5 (patience 2)
Image size	224 × 224	224 × 224

Table 3. Training hyperparameters for Phase 1 (head training) and Phase 2 (fine-tuning).

6. Results and Discussion

6.1 Training Dynamics

Figure 1 and Figure 2 show the training and validation loss/accuracy curves for Phase 1 and Phase 2, respectively. In Phase 1, both training and validation accuracy rapidly converge to near-100% within the first 2 epochs, indicating that the MobileNetV2 features, despite being frozen, are highly discriminative for ASL hand shapes. The training accuracy reached 99.65% and validation accuracy reached 100% by the end of Phase 1.

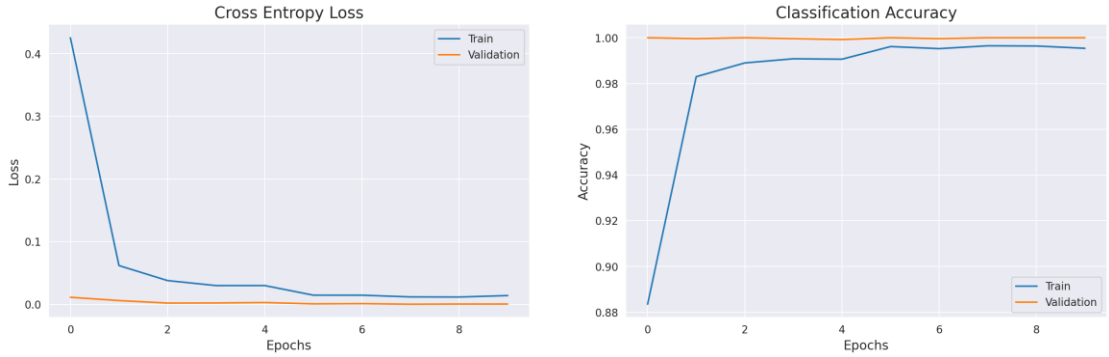


Figure 1. Training vs. Validation Loss and Accuracy — Phase 1 (Classifier Head Training).

In Phase 2, fine-tuning the top layers of the backbone further reduced the validation loss from ~0.001 to below 0.00004, confirming that domain-specific adaptation of the higher-level features provides incremental but consistent improvements. The learning rate decay schedule (ReduceLROnPlateau) contributed to this smooth convergence.

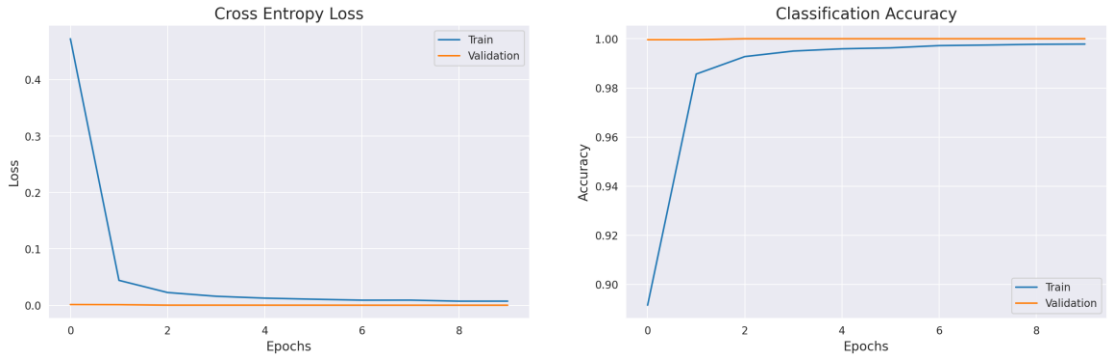


Figure 2. Training vs. Validation Loss and Accuracy — Phase 2 (Fine-Tuning).

6.2 Test Set Performance

The best model (selected by highest validation accuracy during training) was evaluated on the held-out test set of 2,600 images. Table 4 summarizes the overall performance metrics.

Metric	Value
Test Accuracy	99.88%
Macro Precision	99.89%
Macro Recall	99.88%
Macro F1-Score	99.88%
Top-5 Accuracy	100.00%
Test Loss	0.0128

Table 4. Overall performance metrics on the test set (2,600 images, 26 classes).

The model achieves near-perfect performance across all four metrics, with a Top-5 accuracy of 100%, meaning the correct class is always among the top-5 predictions.

6.3 Per-Class Analysis

Figure 3 shows the per-class F1-score for all 26 ASL letters. 23 out of 26 classes achieve a perfect F1-score of 1.00. The only letters with scores marginally below 1.00 are M (F1=0.995), V (F1=0.990), and W (F1=0.990).

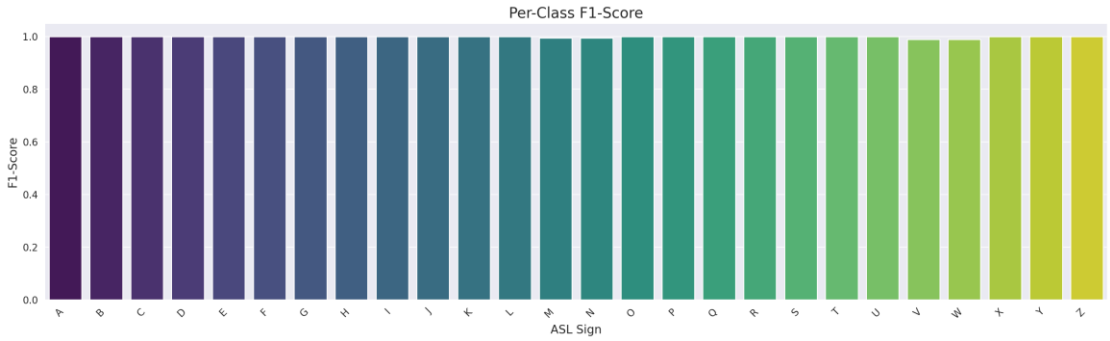


Figure 3. Per-class F1-score for all 26 ASL alphabet classes.

6.4 Confusion Matrix and Error Analysis

Figure 4 shows the full 26×26 confusion matrix evaluated on the test set. The matrix is nearly diagonal, confirming the model's high discriminative ability across all classes. Only three misclassifications were observed across 2,600 test samples.

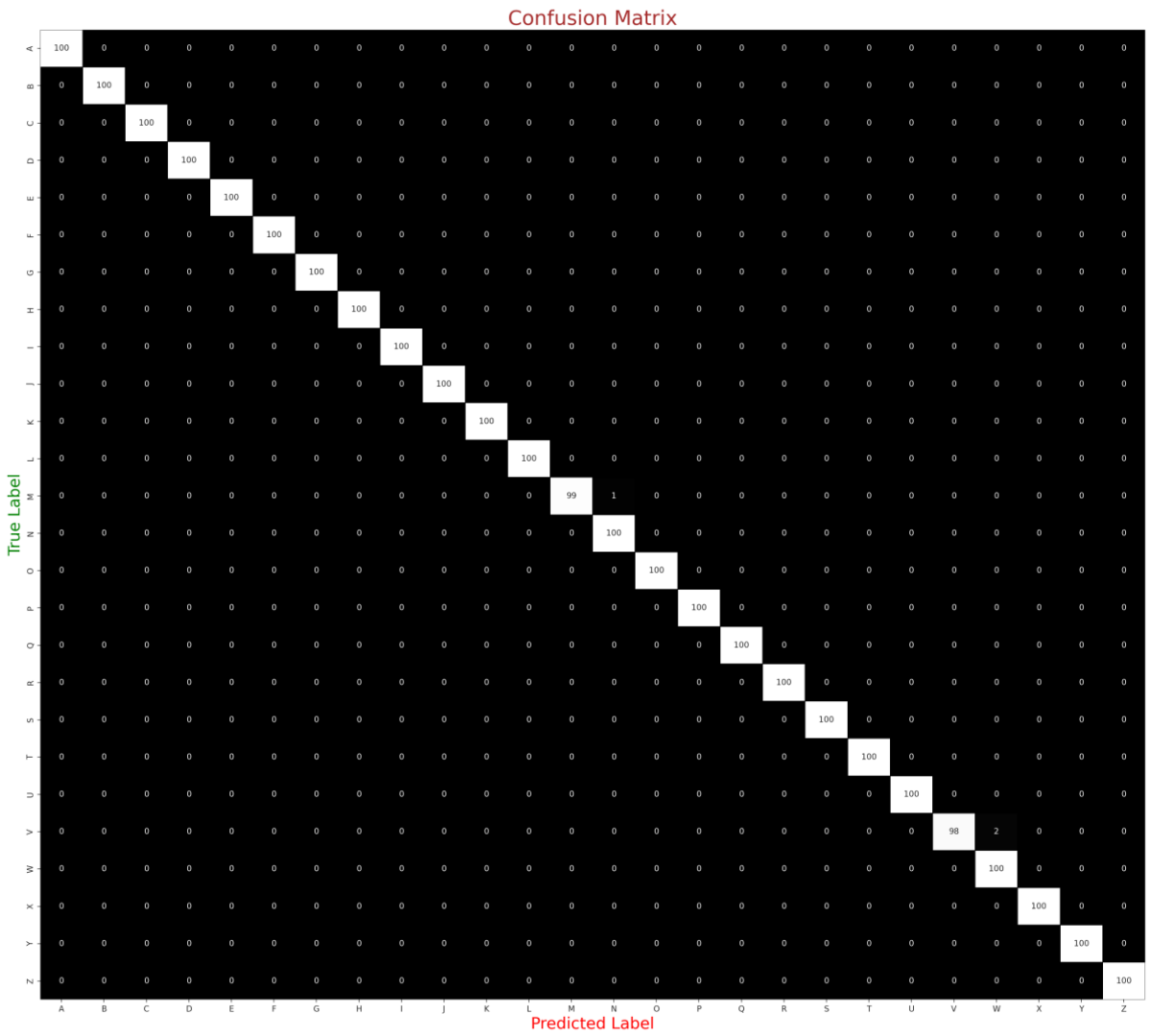


Figure 4. Confusion matrix on the test set (26 classes, 2,600 images).

Table 5 lists all confused pairs identified by the model. The most frequent confusion is V→W (2 instances), which is visually intuitive as both signs involve extended fingers that differ only in subtle hand configuration. The M→N confusion (1 instance) reflects the visual similarity between these two letter forms, which involve overlapping finger placements.

True Label	Predicted As	Count
V	W	2
M	N	1
Total misclassifications		3 / 2,600

Table 5. All misclassified pairs on the test set. Total error rate: 0.12%.

6.5 Per-Class Classification Report

Table 6 presents the full per-class precision, recall, and F1-score for all 26 ASL classes as computed on the 2,600-image test set.

Class	Precision	Recall	F1-Score	Support

A	1.00	1.00	1.00	100
B	1.00	1.00	1.00	100
C	1.00	1.00	1.00	100
D	1.00	1.00	1.00	100
E	1.00	1.00	1.00	100
F	1.00	1.00	1.00	100
G	1.00	1.00	1.00	100
H	1.00	1.00	1.00	100
I	1.00	1.00	1.00	100
J	1.00	1.00	1.00	100
K	1.00	1.00	1.00	100
L	1.00	1.00	1.00	100
M	1.00	0.99	0.995	100
N	0.99	1.00	0.995	100
O	1.00	1.00	1.00	100
P	1.00	1.00	1.00	100
Q	1.00	1.00	1.00	100
R	1.00	1.00	1.00	100
S	1.00	1.00	1.00	100
T	1.00	1.00	1.00	100
U	1.00	1.00	1.00	100
V	1.00	0.98	0.990	100
W	0.98	1.00	0.990	100
X	1.00	1.00	1.00	100
Y	1.00	1.00	1.00	100
Z	1.00	1.00	1.00	100

Table 6. Per-class precision, recall, and F1-score on the test set. Macro average: Precision=1.00, Recall=1.00, F1=1.00.

7. Real-Time Inference Application

To demonstrate practical applicability, I developed a real-time ASL sign detection web application using Streamlit (app_2.py). The application integrates two components:

- MediaPipe Hand Landmarker: Detects hands in the captured image and provides 21 3D landmark coordinates. The bounding box of these landmarks is used to automatically crop the hand region, removing background clutter and ensuring the model receives a well-framed input.
- MobileNetV2 Classifier: The cropped hand image is resized to 224×224, preprocessed, and passed through the trained model. The application displays the predicted sign label, confidence score, and the Top-5 class probabilities as an interactive table.

The Streamlit UI allows users to capture a hand sign via the device camera in real time. The detection and prediction pipeline runs in under one second on a standard CPU, making the system practical for interactive accessibility applications. A 10% padding is added around the raw landmark bounding box to ensure the full hand is captured, as hand edges can be missed by tightly fitted bounding boxes.

Limitations

While the model achieves near-perfect performance on the SignAlphaSet test set, several important limitations must be acknowledged:

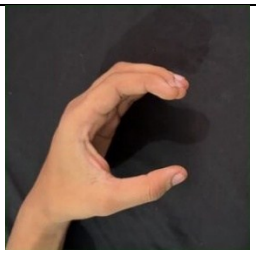
- Controlled background: All training images were captured against a matte black background. Performance may degrade on diverse or cluttered real-world backgrounds without the MediaPipe hand-cropping pipeline.
- Static gestures only: The current model recognizes individual letter signs in isolation. It does not support continuous sign language sentences or dynamic gestures such as movement-based signs (e.g., J and Z involve motion trajectories).
- Dataset diversity: Despite multi-participant data collection, only 3 subjects were included. Greater demographic diversity (age, ethnicity, hand size, hand shape) is needed for broader generalization.
- Similarity confusions: The few observed misclassifications (V/W, M/N) reflect genuine visual ambiguity in the ASL alphabet. Future work could employ attention mechanisms or landmark-based embeddings to improve discrimination.
- No sentence-level recognition: The system processes single signs and does not model temporal context, word boundaries, or grammar—all essential for practical ASL translation.

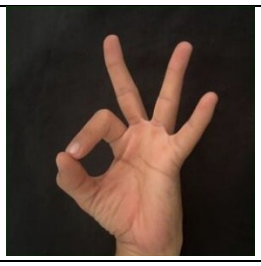
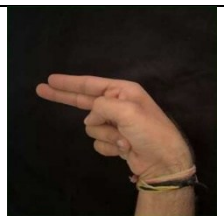
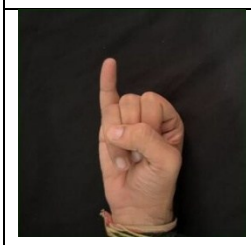
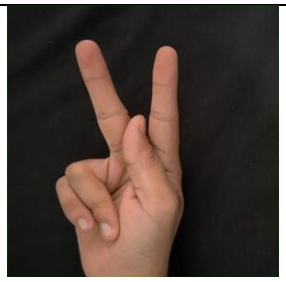
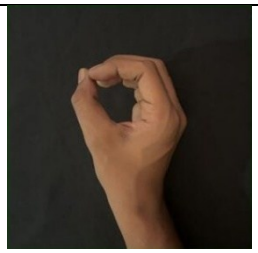
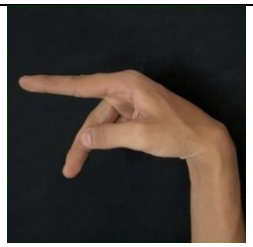
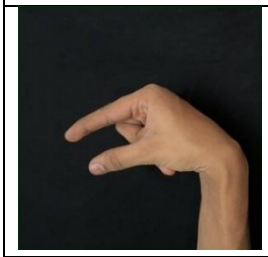
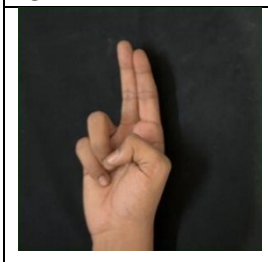
9. Conclusion

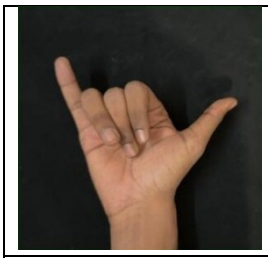
This presents a lightweight, high-performance system for American Sign Language alphabet recognition using transfer learning with MobileNetV2. Trained on the SignAlphaSet dataset and evaluated on a balanced 2,600-image test set, the model achieves 99.88% test accuracy, 99.88% macro F1-score, and 100% Top-5 accuracy. The two-phase training strategy—first adapting the classification head, then fine-tuning the backbone at a low learning rate—proves highly effective and sample-efficient.

Future directions include: (1) expanding the training dataset to include more diverse participants and environmental conditions; (2) extending the model to continuous ASL recognition using sequence models (e.g., LSTM, Transformer); (3) incorporating hand landmark features from MediaPipe as auxiliary inputs to improve discrimination of visually similar signs.

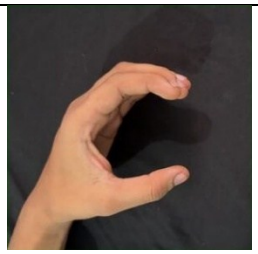
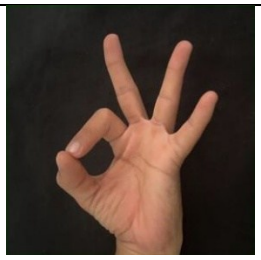
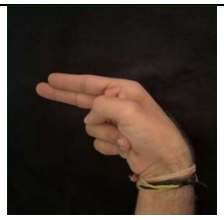
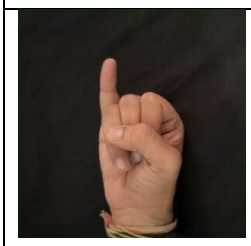
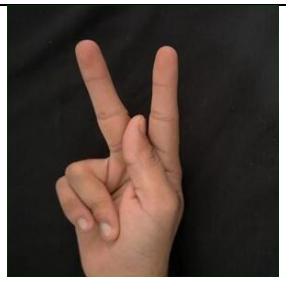
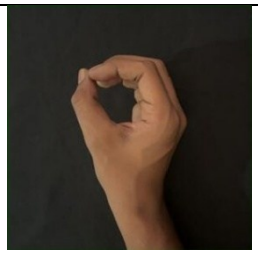
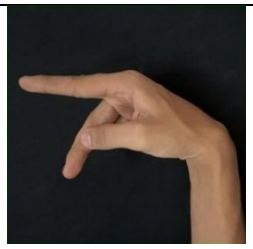
New Pre-processed dataset (26,000) results in Streamlit app (MobilenetV2):

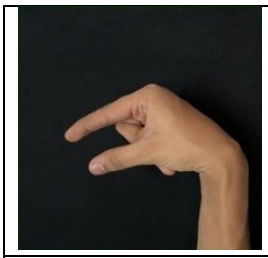
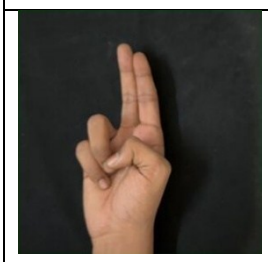
			
A	B	C	D -sometimes fail

			
E	F	G – Failing all times	H
			
I	J -sometimes fail	K	L
			
M – Failing all times	N	O	P
			
Q	R	S – Failing all times	T – Failing all times
			
U	V – Failing most of the time	W	X

			
Y	Z – Failing all times		

New Pre-processed dataset (26,000) results in Streamlit app (Resnet50):

			
A	B	C	D – Failing all times
			
E	F – sometimes fail	G – Failing all times	H
			
I	J	K	L
			
M – Failing all times	N – Failing all times	O	P

			
Q	R	S – Failing sometimes	T – Failing all times
			
U	V	W	X - Failing all times
			
Y	Z – Failing all times		

Reasons for failure (though the results improved a lot):

Most letters that are still failing (like G, M, T, J) have been taken only from the one dataset and lacks variety. Others like S (resembles E) and V (resembles K), D (resembles I) so training with more data could solve the problem.

Code Contribution:
Percentage of Code from Internet Sources: ~45%
Percentage of Code using GenAI: ~50%

Data sources:
https://www.kaggle.com/datasets/grassknoted/asl-alphabet/data

https://data.mendeley.com/datasets/8fmvr9m98w/2

Codes used from:

https://www.kaggle.com/code/sachinpatil1280/hand-sign-multi-class-classification-cnn-97

https://github.com/Dnyana9/tranafer-learning-with-MobileNetV2/tree/main

https://github.com/OmarKhaled1504/AmericanSignLanguage/blob/main/ASL_Transfer_Learning(ResNet50V2).ipynb

https://github.com/mohamed-makrani/Asl-Sign-Language-Recognition

https://github.com/DanielKerger/gesture-detection-and-asl-understanding/tree/main

References

Akash. (2018). ASL Alphabet [Dataset]. Kaggle. https://www.kaggle.com/datasets/grassknoted/asl-alphabet/data

Alsharif, B., et al. (2023). Deep learning technology to recognize American Sign Language alphabet. *Sensors, 23*(18), 7970. https://doi.org/10.3390/s23187970

Benabbas, Y. (2022). American Sign Language (ASL) alphabet dataset (Ver. 2). Mendeley Data. https://data.mendeley.com/datasets/8fmvr9m98w/2

Gerard Aflague Collection. (n.d.). *American Sign Language (ASL) alphabet ABC poster*. https://www.gerardaflaguecollection.com/products/american-sign-language-asl-alphabet-abc-poster.html

Google. (2023). MediaPipe hand landmark detection. https://developers.google.com/mediapipe/solutions/vision/hand_landmarker

Lum, K. Y., Goh, Y. H., & Lee, Y. B. (2020). American Sign Language recognition based on MobileNetV2. *Advances in Science, Technology and Engineering Systems Journal, 5*(6), 481-488. https://www.astesj.com/v05/i06/p57/

Otsu, N. (1979). A threshold selection method from gray-level histograms. IEEE TSMC, 9(1), 62–66.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 4510–4520.

Zhang, Y., & Jiang, X. (2024). Recent advances on deep learning for sign language recognition. Computer Modeling in Engineering & Sciences, 139(1), 1–40.

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Ubowejewski, E., Song, C., ... & Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. arXiv preprint arXiv:1906.08172.